

Manual de Instruções para a Interface e Relatório

Autor(es): Gabriel Sanches da Silva

E-mail(s): gabrielss2403@usp.br

Resumo

Este documento é dividido em duas etapas principais: a primeira dela é um manual de instruções de como usar a interface, definindo arquitetura de arquivos e como interpretar resultados; a segunda é voltada para pesquisadores e/ou desenvolvedores que precisam enxergar a parte mais técnica da interface, dos códigos utilizados, e do modelo de otimização elaborado.

Parte I

Relatório e Detalhes

1 Modelo Principal

1.1 Criação do Modelo

O modelo de alocação de aulas é definido como:

$$\text{Minimizar} \quad \text{Função Objetivo} \quad Z$$

1.2 Conjuntos e Índices

- A é o conjunto das aulas a , incluindo aulas que não possuem horário definido e sem inscritos.
- A^t é o conjunto das aulas a de uma turma/disciplina t , incluindo aulas que não possuem horário definido e sem inscritos.
- A^{tt} é o conjunto das aulas a de uma turma/disciplina t , sem incluir aulas que não possuem horário definido e sem inscritos.
- A^c é o conjunto das aulas a que são ministradas ao curso c , incluindo aulas que não possuem horário definido e sem inscritos.
- S é o conjunto das salas s disponíveis para alocação.
- T é o conjunto das turmas/disciplinas t que serão ministradas no semestre.

1.3 Dados de Entrada

- tam_t é um vetor de t valores, um para cada turma/disciplina, onde cada posição tem o tamanho da turma t .
- cap_s é um vetor de s valores, um para cada sala, onde cada posição tem a capacidade máxima da sala s .
- $DIS_{s,sl}$ é uma matriz com as distâncias de uma sala s a uma sala sl .
- $pref_s$ é um vetor binário de valor 1 quando a sala s é preferencialmente vazia, e 0 caso contrário.

1.4 Dados de Preprocessamento

- $seguidas_t$ é um vetor de t valores, um para cada turma/disciplina. O valor de $seguidas_t$ é 1 quando a turma t não possui aulas seguidas, e 2 caso contrário.
- Θ_{aal} é uma matriz binária de valor 1 quando as aulas a e al possuem conflito de horário, e 0 caso contrário.
- η_{as} é uma matriz binária de valor 1 quando a aula a pode ser alocada na sala s . Essa matriz considera o tamanho da sala e o tamanho da turma, proibição manual, fixação, e uso de laboratório. Em outras palavras:
 - Quando a aula a não couber em uma sala s , $\eta_{as} = 0$, ou 1 caso contrário.
 - Quando a aula a estiver proibida de ser alocada em uma sala s , mesmo que normalmente ela pudesse ser, $\eta_{as} = 0$, ou 1 caso contrário.

- Quando a aula a estiver fixada em uma sala s , $\eta_{as} = 1$, ou 0 caso contrário.
- Quando a aula a for uma aula de laboratório, ela pode apenas ser alocada em salas s que são laboratórios e que ela couber. Logo, o η_{as} dela só pode ser 1 quando a couber em s e s ser uma sala de laboratório.
- De forma semelhante, caso a aula a não seja de laboratório, o η_{as} dela só poder 1 quando a couber em s e s não ser uma sala de laboratório.
- USO_{as} é uma matriz com os percentuais de preenchimento de uma aula a em uma sala s .

1.5 Variáveis de Decisão

- $x_{as} \in \{0, 1\}$: variável binária que assume o valor 1 se a aula a é alocada à sala s , e 0 caso contrário.
- $y_t \geq 0$: variável inteira que contabiliza o número de trocas de sala de uma turma t .
- $c_{st} \in \{0, 1\}$: variável binária que assume o valor 1 se a sala s é utilizada pela turma t .
- $w_{cs} \in \{0, 1\}$: variável binária que assume o valor 1 se o curso c tem ao menos uma aula na sala s .
- $v_{cssl} \in \{0, 1\}$: variável binária que assume o valor 1 se o curso c tem ao menos uma aula nas salas s e sl .
- $z_{as} \in \{0, 1\}$: variável binária que assume o valor 1 se a aula a , ao ser alocada à sala s , ultrapassa uma certa quantia do espaço disponível.

1.6 Função Objetivo

A função objetivo é dada por:

$$\begin{aligned}
\min Z = & \text{peso}_x \left(\sum_{a \in A} \sum_{s \in S} USO_{as} x_{as} \right) + \text{peso}_y \sum_{t \in T} seguidas_t y_t \\
& + \text{peso}_v \left(\sum_{c \in C} \sum_{s \in S} \sum_{sl \in S, s \neq sl} DIS_{s,sl} v_{cssl} \right) \\
& + \text{peso}_z \left(\sum_{a \in A} \sum_{s \in S} z_{as} \right) + \text{peso}_{\text{pref}} \left(\sum_{a \in A} \sum_{s \in S} \text{pref}_s x_{as} \right).
\end{aligned} \tag{1}$$

1.7 Restrições

(1) Restrições de alocação:

$$\sum_{s \in S} x_{as} = 1, \quad \forall a \in A \text{ tal que } \text{start}_a \neq 0 \text{ e } \text{tam}_t \neq 0. \tag{2}$$

$$\sum_{s \in S} x_{as} = 0, \quad \forall a \in A \text{ tal que } \text{start}_a = 0 \text{ ou } \text{tam}_t = 0. \tag{3}$$

(2) Restrições de alocação:

$$x_{as} \leq \eta_{as}, \quad \forall a \in A, s \in S. \tag{4}$$

(3) Restrições de conflito de horário:

$$x_{as} + x_{al,s} \leq 1, \quad \forall a, al \in A, \forall s \in S \text{ tal que } \theta_{aal} = 1. \tag{5}$$

Uso de Salas por Turma/Disciplina

Para cada turma/disciplina t :

$$\sum_{a \in A_{tt}[t]} x_{as} \leq |A_{tt}[t]| \cdot c_{st}, \quad \forall s \in S, \forall t \in T \quad (6)$$

$$y_t \geq \sum_{s \in S} c_{st}, \quad \forall t \in T \quad (7)$$

Restrição de Superlotação

Se houver um limite de ocupação α , então para cada aula a e sala s :

$$\text{tam}_t(a) \cdot x_{as} \leq \alpha \cdot \text{cap}_s + z_{as} \cdot \text{cap}_s \quad \forall a \in A, \forall s \in S \quad (8)$$

Relação entre w_{cs} e x_{as}

Para cada sala s , curso c e aula a pertencente ao curso c :

$$x_{as} \leq w_{cs}, \quad \forall a \in A_c[c], \forall c \in C, \forall s \in S \quad (9)$$

Relação entre w_{cs} e v_{cssl}

Para cada curso c e pares de salas s, sl com $s \neq sl$:

$$2v_{cssl} \leq w_{cs} + w_{csl} \quad \forall c \in C, \forall s \in S, \forall sl \in S, sl \neq s \quad (10)$$

$$v_{cssl} \geq w_{cs} + w_{csl} - 1 \quad \forall c \in C, \forall s \in S, \forall sl \in S, sl \neq s \quad (11)$$

Parte II

Explicação do código e dos algoritmos

Nesta seção, serão explicados os funcionamentos de algumas funções e algoritmos utilizados para a criação da interface, a verificação dos dados e a execução do modelo.

2 interface_final.py

O arquivo `interface_final.py` é o responsável por criar a interface, ou seja, ele é o *script* capaz de manipular arquivos e chamar os demais scripts que serão descritos.

2.1 Classe Tooltip

A classe `Tooltip` é chamada em algumas janelas da interface para mostrar textos ao sobrevoar o mouse em certos lugares da janela. Em particular, ela foi utilizada para explicar as variáveis e pesos do modelo de otimização.

2.2 Função de Padronização de DataFrame

A função de padronização do dataframe (`padroniza_dataframe`) é chamada durante a concatenação dos elencos das aulas, isto é, durante o processo de "Criar a Planilha com os dados das Aulas". Como seu nome descreve, a função tenta padronizar cada arquivo em um determinado formato, alterando o nome de certas colunas e até criando outras.

Seus parâmetros são:

- `file_name` : *uma string contendo o caminho para uma planilha de elenco que será padronizada.*
Dentro da padronização, a função:
 - Retira quebras de linhas nos nomes das colunas.
 - Renomeia algumas delas, como a de salas e de observações.
 - Insere colunas extras, como as de Horário 3 e 4, e as colunas de Proibir Horário.
 - Filtra as disciplinas que serão alocadas no ICMC, marcadas por um 'X' na planilha.
 - Verifica se algum dado foi preenchido erroneamente, como um horário fora do padrão, ou se a turma de uma disciplina não foi identificada.

2.3 Função do Primeiro e Segundo Botão da Interface

A função `planilha_dep` é a função utilizada para criar as janelas do primeiro e segundo botão da interface. Por seu funcionamento ser semelhante, utiliza-se uma variável booleana `jupiter` e estruturas condicionais para criar cada janela.

A função cria diversas variáveis para salvar o caminho de arquivos de planilhas selecionados pelo usuário, assim dando início a concatenação dos mesmos. Além disso, também são definidos labels e botões que são exibidos na janela, contribuindo para um uso mais claro da interface. Com os arquivos selecionados, o usuário pode prosseguir para o próximo passo e chamar a função `concat_df`.

2.4 Função de leitura de DataFrame

A função `ler_df` é uma função genérica que lê um arquivo dado pelo seu parâmetro `caminho_arquivo` como um DataFrame do Pandas. Ela é utilizada pela função `concat_df` para ler tantos os arquivos de planilhas do elenco quanto os dados do Júpiter, sendo capaz de diferenciar a leitura de arquivos `.xlsx` de `.csv`.

2.5 Função que concatena planilhas

A função `concat_df` é responsável por fazer a concatenação dos elencos e dados do JúpiterWeb. Seus parâmetros são:

- `SME, SMA, SCC, SSC`: caminhos dos arquivos dos departamentos principais.
- `salas`: caminho do arquivo de dados das salas (usado apenas se `jupiter=False`).
- `nome_arquivo`: nome do arquivo final a ser gerado.
- `ano`: ano dos dados (usado apenas se `jupiter=False`).
- `jupiter`: booleano, indica se é para criar a base do JúpiterWeb (True) ou das aulas (False).
- `outros`: caminho(s) dos arquivos dos outros institutos (str ou list).

Como a função `planilha_dep`, ela executa um algoritmo baseado se os arquivos sendo concatenados são os elencos das aulas ou os dados do JupyterWeb. Para o caso dos elencos, o algoritmo funciona lendo um dos arquivos, procurando pela linha contendo os cabeçalhos corretos e chamando a função de padronização `padroniza_dataframe`. Para o caso dos dados do JupyterWeb, o algoritmo apenas lê os arquivos e salva os três primeiros dígitos das disciplinas listadas neles.

Ao fim de ambos os algoritmos, um novo arquivo `.xlsx` é criado utilizando o nome obtido das planilhas, isto é, cada aba do novo arquivo possui o nome do departamento correspondente.

2.6 Função do Terceiro e Quarto Botão da Interface

A função `base_dados` é a função chamada para o terceiro e o quarto botão da interface. Ela é responsável por criar a janela para selecionar os arquivos necessários para compor a base de dados utilizada pelo modelo. Semelhante a outras funções, ela possui um parâmetro booleano chamado `pior_caso`, que altera como a janela em questão será aberta, e quais variáveis e funções serão utilizadas dentro do código.

Caso o usuário aperte o terceiro botão e selecione os arquivos necessários, o script `jupiter_sheet_maker.py` será chamado. Caso o usuário aperte o quarto botão e selecione os arquivos necessários, a função `base_pior_caso` é chamada.

2.7 Função que cria a Base de Dados do Modelo

A função `base_pior_caso` é chamada quando o usuário precisa criar a base de dados de pior caso, possuindo os seguintes parâmetros e retornos:

- `df1`: base de dados mais recente (dict de DataFrames por sheet)
- `df2`: base de dados mais antiga (dict de DataFrames por sheet)
- `sheets`: lista de nomes das planilhas a serem analisadas
- Retorno: `df1` atualizado com os maiores valores de inscritos e ano dos dados.

2.8 Função do Quinto Botão da Interface

A função `execute` é a função responsável por criar a janela aberta quando o usuário clicar no quinto botão da interface. Nela, o usuário seleciona uma base de dados para ser analisada antes de executar o modelo.

Caso o usuário clique no botão de verificação dos dados, a função `roda_script` para rodar o script `verificar_dados.py` é chamada. Caso o usuário clique no botão de executar o modelo com os dados, a função `Nova_edit_config` é chamada para escolher o modelo a ser executado.

2.9 Função que chama um script para ser executado

A função `roda_script` é a função utilizada para chamar os scripts de verificar os dados e de executar o modelo. Seus parâmetros são:

- `script`: nome do script Python a ser executado
- `nome`: caminho do arquivo da base de dados
- `peso_x`, `peso_y`, `peso_v`, `peso_z`, `alpha`, `pref`: parâmetros do modelo a serem passados ao script

Após chamar o script, todas as saídas deles são replicadas em uma janela da interface. Dessa forma, quando ocorre algum erro no script, o erro pode ser apresentado diretamente ao usuário, seja ele um erro com tratamento especial ou não.

2.10 Função que cria a janela para escolher o Modelo

A função `Nova_edit_config` é a função responsável por criar uma janela na interface para que o usuário selecione o modelo para a alocação. Após definir as caixas de marcação para o usuário selecionar os pesos do modelo, é definida uma função interna `atualizar_estado` que ativa e desativa cada caixa, dependendo de qual foi selecionada. Depois disso, a função repete um processo de criar uma linha de separação, uma legenda explicativa, uma `Tooltip` ligada à legenda, e as duas caixas correspondentes.

Por fim, são definidos mais dois botões: um que executa o modelo utilizando os parâmetros utilizados pelo autor (configurações padrão), e outro que executa o modelo utilizando os parâmetros personalizados do usuário.

2.11 Função do Sexto Botão da Interface

A função `gerar_relatorios` é responsável por criar um submenu na interface, para que o usuário selecione um tipo de relatório a ser gerado. Isso é feito definindo os botões do submenu na janela, que chamam a função `menu_relatorios` com o parâmetro que define o relatório que será gerado.

2.12 Função que chama as funções de gerar relatórios

A função `menu_relatorios` cria uma janela com um botão para selecionar um arquivo de solução do modelo, que será a base do relatório. Quando o usuário escolher o arquivo e apertar no botão de geração, um algoritmo será executado para montar as variáveis contendo as componentes das tabelas, isto é, colunas de horários e linhas com os dias da semana.

Seu parâmetro `func` é uma string que identifica qual função de geração será utilizada.

2.13 Função da Visualização Completa

A função `visualizacao_completa` é responsável por criar o arquivo Excel da Visualização Completa da Solução, utilizando as linhas e colunas definidas anteriormente, e os dados da solução. Seus parâmetros são:

- `dfv`: DataFrame com os dados de alocação das aulas
- `salas1`: lista de salas utilizadas
- `horarios`: lista dos horários disponíveis
- `sala_colunas`: número de colunas de horários
- `dias_semana`: lista dos dias da semana

O algoritmo que chama a função genérica de preenchimento foi feito para colocar todas as alocações feitas pelo modelo, por isso a visualização é chamada de completa.

2.14 Função da Visualização por Curso

A função `visualizacao_curso` é muito semelhante à anterior, incluindo seus parâmetros. Contudo, o algoritmo utilizado para o preenchimento difere para que o arquivo criado tenha várias abas, onde cada aba corresponde às alocações feitas para um determinado curso do ICMC.

2.15 Função da Visualização por Departamento

A função `visualizacao_dep` é muito semelhante à anterior, incluindo seus parâmetros. Contudo, o algoritmo utilizado para o preenchimento difere para que o arquivo criado tenha várias abas, onde cada aba corresponde às alocações feitas para um determinado departamento do ICMC.

2.16 Função dos arquivos para a Intranet

A função `planilhas_intranet` é responsável por criar os arquivos que serão enviados ao sistema da intranet. Nessa função, definimos algumas outras funções para padronizar e extrair os horários de cada aula dos dados da solução.

Com os dados padronizados, cria-se um arquivo de todas as alocações, e depois criam-se vários arquivos no mesmo formato do primeiro, mas um para cada sala disponível do instituto.

2.17 Função Genérica de Gerar Relatório

A função `funcao_visualizacao` é responsável por fazer o preenchimento da planilha do Excel, seguindo os dados fornecidos na hora de gerar um relatório. Ela é a função genérica chamada nas funções anteriores para criar os preenchimentos. Seus parâmetros são:

- `start_row`, `start_column`, `end_row`, `end_column`: coordenadas para mesclagem e posicionamento da tabela da sala
- `start`: linha inicial para os dias da semana
- `sala`: nome ou número da sala a ser preenchida
- `dias_semana`: lista dos dias da semana
- `horarios`: lista dos horários disponíveis
- `aula`: lista dos horários das aulas alocadas na sala
- `disciplina`: lista dos códigos das disciplinas alocadas
- `ws`: worksheet do Excel a ser preenchida
- `aulas`: contador de aulas já alocadas

2.18 Função para analisar o espaços livres

A função `analise_vazios()` é responsável por abrir uma janela secundária na interface gráfica, permitindo ao usuário selecionar os arquivos necessários para a geração da análise dos espaços livres após a alocação do modelo. Essa janela também possibilita a definição dos nomes dos arquivos de saída que conterão tanto a visualização quanto a planilha resultante da análise.

Após o usuário selecionar os arquivos necessários e fornecer o nome dos novos arquivos, a função `criar_visualizacao_de_vazias` é chamada.

2.19 Função que cria a visualização de espaços livres

A função `criar_visualizacao_de_vazias` é responsável por criar os dois "relatórios" de espaços livres, no caso, o arquivo de visualização e o arquivo com os dados desses espaços livres. Seus parâmetros são:

- `visu`: caminho do arquivo de visualização completa da solução do modelo;
- `salas`: caminho do arquivo contendo os dados das salas;
- `caminho_arquivo`: nome do arquivo de saída da visualização dos espaços livres;
- `caminho_arquivo1`: nome do arquivo de saída da planilha de espaços livres.

Ela segue um algoritmo muito semelhante ao da função `visualizacao_completa`, passando por cada célula do arquivo da Visualização Completa da Solução, alterando a cor e limpando o conteúdo das células mescladas.

2.20 Função do Sétimo Botão da Interface

A função `preencher_planilha_dados` é responsável por preencher as planilhas com os dados das alocações, ou seja, as planilhas com os elencos das aulas e a base de dados utilizada pelo modelo.

Inicialmente, a função coleta uma resposta do usuário com uma caixa de mensagem, que altera a janela para selecionar os arquivos que serão preenchidos. Caso o usuário queira preencher as planilhas com todos os dados da solução, uma janela é aberta para o usuário adicionar as planilhas que serão preenchidas; caso o usuário queira apenas fixar algumas alocações, isto é, manter apenas alguns dos dados da solução, uma janela é aberta para que o usuário selecione as fixações.

2.21 Função para o usuário escolher as alocações

A função `escolhas_preenchimento` é responsável por criar a janela que permite ao usuário escolher quais alocações do modelo serão mantidas. Com um loop para gerar as caixas de seleção, todas as aulas que foram alocadas são listadas com seus dados e com sua respectiva caixa. Ao clicar no botão no final da janela, um algoritmo é chamado para preencher a base de dados do modelo apenas com dados selecionados pelo usuário. Seus parâmetros são:

- `file_path_sol`: caminho da planilha de solução do modelo
- `file_path_base`: caminho da planilha de base de dados do modelo

Com os dados selecionados, a função `preenchimento` é chamada para realizar o preenchimento "parcial" dos dados.

2.22 Função para preencher com todos os dados da solução

A função `preenchimento` é responsável por preencher as planilhas de dados que o usuário escolheu. Seus parâmetros são:

- `lista_elenco`: lista de caminhos dos arquivos de elenco a serem preenchidos
- `file_path_sol`: caminho da planilha de solução do modelo
- `file_path_base`: caminho da planilha de base de dados do modelo
- `preencher_parcial`: booleano indicando se o preenchimento é parcial (True) ou não (False)

A função preencherá os arquivos de acordo com a necessidade, isto é, se a lista de elencos tiver apenas um arquivo, apenas aquele arquivo será preenchido com os dados da solução. O preenchimento das planilhas de elenco também são diferentes do preenchimento para a base de dados do modelo. Ao final da função, todos os arquivos criados são listados para o usuário.

3 jupiter sheet maker.py

O script `jupiter sheet maker.py` é um programa Python desenvolvido para processar e consolidar informações de múltiplas planilhas Excel relacionadas ao sistema acadêmico Júpiter. O programa realiza a integração de dados de salas, vagas por curso, ingressantes e espelho de matrículas, gerando uma planilha consolidada com informações sobre disponibilidade de vagas por disciplina.

3.1 Arquivos de Entrada

O script recebe 5 arquivos como argumentos de linha de comando:

1. **sys.argv[1]**: Arquivo Excel com múltiplas planilhas contendo dados de salas e disciplinas por departamento (SME, SMA, SCC, SSC, Outros)
2. **sys.argv[2]**: Arquivo Excel do sistema Júpiter com informações de vagas disponíveis por disciplina
3. **sys.argv[3]**: Planilha de ingressantes
4. **sys.argv[4]**: Planilha espelho (com dados de inscritos)
5. **sys.argv[5]**: Nome do arquivo de saída

3.2 Fluxo de Processamento

3.2.1 Carregamento dos Dados

```
df_completo = pd.ExcelFile(sys.argv[1])
sheet_names = df_completo.sheet_names
salas = pd.read_excel(sys.argv[1], sheet_name=sheet_names[0])
df = pd.read_excel(sys.argv[1], sheet_name=sheet_names[1:])
```

Nesta etapa, o programa:

- Carrega o arquivo principal com todas as planilhas
- Extrai a primeira planilha como dados de salas
- Carrega as demais planilhas (SME, SMA, SCC, SSC, Outros) como um dicionário de DataFrames

3.2.2 Normalização de Códigos de Disciplina

O script realiza normalização dos códigos de disciplina em três arquivos diferentes (ingressantes, espelho e planilhas principais). A normalização consiste em:

1. **Remoção de espaços**: Remove espaços em branco dos códigos de disciplina
2. **Padronização do formato**: Garante que todos os códigos sigam o formato CODIGO-TURMA

```
for d in range(len(ingressantes['Disciplina (codigo)'])):
    if ' ' in str(ingressantes.loc[d, 'Disciplina (codigo)']):
        ingressantes.loc[d, 'Disciplina (codigo)'] =
            str(ingressantes.loc[d, 'Disciplina (codigo)']).replace(' ', '')
    if '-' not in str(ingressantes.loc[d, 'Disciplina (codigo)']):
        ingressantes.loc[d, 'Disciplina (codigo)'] =
            f"{ingressantes.loc[d, 'Disciplina (codigo)']}-{int(ingressantes.loc[d, 'T'
```

Esta seção processa cada linha da planilha de ingressantes, formatando os códigos de disciplina para o padrão CODIGO-TURMA.

Tratamento de Erros O código implementa tratamento robusto de erros usando blocos `try-except`:

```
except KeyError as e:
    coluna = str(e).strip('\ ')
    msg = f"A coluna '{coluna}' não foi encontrada no arquivo {os.path.basename(sys.argv[1])}"
    print(msg, file=sys.stderr)
    sys.exit(4)
```

Quando uma coluna esperada não é encontrada, o programa exibe mensagem de erro detalhada e encerra com código de saída específico.

3.3 Processamento de Vagas

3.3.1 Preenchimento de Valores Nulos

```
for sheet in sheet_names_jupyter[0:]:
    vagas[sheet] = vagas[sheet].fillna(0)
```

Substitui valores vazios (NaN) por zero em todas as planilhas de vagas do Jupyter.

3.3.2 Cálculo de Vagas por Disciplina

Para as planilhas SME, SMA, SCC e SSC (índices 1 a 4):

1. **Normalização de colunas:** Remove quebras de linha dos nomes das colunas
2. **Formatação de disciplinas:** Padroniza códigos tanto nas planilhas de vagas quanto nas principais
3. **Identificação de colunas de vagas:** Localiza as colunas de diferentes tipos de vagas:
 - Vagas obrigatórias
 - Vagas eletivas
 - Vagas optativas livres
 - Vagas especiais

3.3.3 Soma de Vagas

```
if df[sheet].loc[d, 'Disciplina (codigo)'] in vagas[sheet]['Disciplina'].tolist():
    index = (vagas[sheet]['Disciplina'].tolist()).index(df[sheet].loc[d, 'Disciplina'])
    df[sheet].loc[d, 'Vagas por disciplina'] = 0
    for i in range(len(lista2)):
        df[sheet].loc[d, 'Vagas por disciplina'] += vagas[sheet].loc[index, colunas[i]]
```

Para cada disciplina:

- Verifica se existe na planilha de vagas do Jupyter
- Localiza o índice correspondente
- Soma todas as categorias de vagas (obrigatórias, eletivas, optativas e especiais)
- Se não encontrada, define vagas como 0

3.4 Ajustes Especiais de Vagas

O script verifica a coluna **Observações** para aplicar ajustes adicionais:

3.4.1 Adição de Ingressantes

```
if "Ingressantes" in str(df[sheet].loc[d, 'Observacoes']):
    if df[sheet].loc[d, 'Disciplina (codigo)'] in ingressantes['Disciplina (codigo)']:
        index = (ingressantes['Disciplina (codigo)'].tolist()).index(df[sheet].loc[d,
        df[sheet].loc[d, 'Vagas por disciplina'] += ingressantes.loc[index, 'Ingressantes'])
```

Se a observação contém a palavra “Ingressantes”, adiciona o número de ingressantes ao total de vagas.

3.4.2 Adição de Espelho

```
if "Espelho" in str(df[sheet].loc[d, 'Observacoes']):
    if df[sheet].loc[d, 'Disciplina (codigo)'] in espelho['Disciplina (codigo)'].tolist():
        index = (espelho['Disciplina (codigo)'].tolist()).index(df[sheet].loc[d, 'Disciplina (codigo)'])
        df[sheet].loc[d, 'Vagas por disciplina'] += espelho.loc[index, 'Inscritos']
```

Se a observação contém a palavra “Espelho”, adiciona o número de inscritos do espelho ao total de vagas.

3.5 Processamento da Planilha “Outros”

A planilha “Outros” (índice 5) recebe tratamento especial:

1. Normalização dos códigos de disciplina (similar às outras planilhas)
2. Processamento de vagas considerando as planilhas do Júpiter de índice 4 em diante
3. Aplicação dos mesmos ajustes de ingressantes e espelho
4. Tratamento especial para valores NaN: mantém como 0 se não encontrada

3.6 Geração do Arquivo de Saída

3.6.1 Estruturação dos Dados

```
dfs = {
    "Salas": salas,
    "SME": df['SME'],
    "SMA": df['SMA'],
    "SCC": df['SCC'],
    "SSC": df['SSC'],
    "Outros": df['Outros']
}
```

Organiza os DataFrames processados em um dicionário com nomes das planilhas.

3.6.2 Escrita do Arquivo Excel

```
file_path = os.path.join(os.getcwd(), "Saidas da Interface", "Planilhas de Dados", file_name)

with pd.ExcelWriter(file_path, engine='openpyxl') as writer:
    for sheet_name, df in dfs.items():
        df.to_excel(writer, sheet_name=sheet_name, index=False)
```

Gera arquivo Excel no diretório **Saidas da Interface/Planilhas de Dados** com todas as planilhas processadas.

3.7 Tratamento de Erros na Escrita

3.7.1 Erro de Permissão

```
except PermissionError as e:
    if e.errno == 13:
        traceback.print_exc(file=sys.stderr)
        sys.exit(2)
```

Captura erros quando o arquivo está aberto ou bloqueado (código de saída 2).

3.7.2 Erros Gerais

```
except Exception as e:
    traceback.print_exc(file=sys.stderr)
    sys.exit(1)
```

Captura qualquer outro erro, imprime o traceback completo e encerra com código 1.

3.8 Códigos de Saída

O programa utiliza diferentes códigos de saída para indicar tipos específicos de erro:

Código	Significado
0	Execução bem-sucedida
1	Erro geral não especificado
2	Erro de permissão (arquivo aberto/bloqueado)
3	Outro erro de permissão
4	Coluna não encontrada em algum arquivo de entrada

Tabela 1: Códigos de saída do programa

3.9 Dependências

O script requer as seguintes bibliotecas Python:

- **pandas**: Manipulação de planilhas Excel e DataFrames
- **openpyxl**: Engine para leitura/escrita de arquivos .xlsx
- **os**: Operações de sistema de arquivos
- **sys**: Acesso a argumentos da linha de comando
- **traceback**: Formatação de mensagens de erro

3.10 Resumo do Fluxo de Execução

1. Carregamento de todos os arquivos de entrada
2. Normalização dos códigos de disciplina em todos os arquivos
3. Preenchimento de valores nulos nas planilhas de vagas
4. Para cada departamento (SME, SMA, SCC, SSC):
 - Cálculo da soma de vagas (obrigatórias + eletivas + optativas + especiais)
 - Adição de vagas de ingressantes (se aplicável)

- Adição de vagas do espelho (se aplicável)
5. Processamento especial da planilha “Outros”
 6. Geração do arquivo Excel consolidado
 7. Tratamento de erros e retorno de código de saída apropriado

Algorithm 1 Padronização de Dataframe

```
1: function PADRONIZA_DATAFRAME(file_name, header_row, ano)
2:           ▷ Função que padroniza os dataframes para terem o mesmo formato
3:   df ← ler arquivo Excel file_name com cabeçalho na linha header_row
4:   for cada coluna col no dataframe do
5:     if contém “\n” no nome da coluna then
6:       renomear coluna substituindo “\n” por espaço
7:     end if
8:     if “Sala” está no nome da coluna then
9:       renomear coluna para “Sala”
10:    end if
11:  end for
12:  for cada linha idx no dataframe do
13:    remover espaços da coluna “Deve ser alocada no ICMC?”
14:    if valor é “x” then
15:      alterar valor para “X”
16:    end if
17:  end for
18:  filtrar dataframe mantendo apenas linhas com “X” na coluna “Deve ser alocada no ICMC?”
19:  if coluna “Horário 3” não existe then
20:    inserir coluna “Horário 3” após “Horário 2” com valores vazios
21:  end if
22:  if coluna “Horário 4” não existe then
23:    inserir coluna “Horário 4” após “Horário 3” com valores vazios
24:  end if
25:  if existe coluna “observações” then
26:    renomear para “Observações”
27:  end if
28:  if coluna “Observações” não existe then
29:    inserir coluna “Observações” após “Horário 4” com valores vazios
30:  end if
31:  headers ← nomes das colunas do dataframe
32:  preencher valores vazios de “Utilizará laboratório?” com “Não”
33:  turmas0 ← índices das linhas com turma vazia
34:  if turmas0 não está vazio then
35:    exibir aviso com números das linhas problemáticas
36:    return None
37:  end if
38:  inserir coluna “Vagas por disciplina” no final
39:  inserir coluna “Ano dos dados” com valor ano no final
40:  for cada coluna header em ['Horário 1', 'Horário 2', 'Horário 3', 'Horário 4'] do
41:    for cada linha d no dataframe do
42:      if contém traço anormal “-” then
43:        substituir por traço normal “.”
44:      end if
45:      if horário não vazio E não contém “-” E não é string vazia then
46:        exibir aviso de formato incorreto
47:        return None
48:      end if
49:      if horário não vazio E mais de 2 partes ao dividir por “-” then
50:        exibir aviso de múltiplos traços
51:        return None
52:    end if
```

Algorithm 2 Padronização de Dataframe (Parte 2)

```
53:         if contém “\n” no horário then  
54:             exibir aviso de quebra de linha  
55:             return None  
56:         end if  
57:     end for  
58: end for  
59:     inserir 4 colunas de proibição: “Proibir Horário 1” a “Proibir Horário 4”  
60:     return df  
61: end function
```

Algorithm 3 Classe Tooltip - Sistema de Dicas Visuais

```
1: Classe: Tooltip
2: Propósito: Exibir textos explicativos ao sobrevoar o mouse em widgets
3: Parâmetros do Construtor:
4:   • widget: componente da interface
5:   • text: mensagem explicativa

6: function CONSTRUTOR(widget, text)
7:   self.widget ← widget
8:   self.text ← text
9:   self.tooltip_window ← null
10:                                     ▷ Vincular eventos de entrada e saída do mouse
11:   widget.bind("<Enter>", self.mostrar_tooltip)
12:   widget.bind("<Leave>", self.ocultar_tooltip)
13: end function

14: function MOSTRAR_TOOLTIP(evento)
15:   if janela do tooltip já existe then
16:     return                                     ▷ Evita criação múltipla
17:   end if
18:   posX ← posiçãoX(widget) + 20 pixels
19:   posY ← posiçãoY(widget) + 20 pixels
20:   criar janela flutuante (Toplevel)
21:   configurar janela sem bordas
22:   posicionar janela em (posX, posY)
23:   criar rótulo com texto explicativo
24:   configurar estilo: fundo amarelo claro, borda sólida
25:   empacotar rótulo na janela
26: end function

27: function OCULTAR_TOOLTIP(evento)
28:   if janela do tooltip existe then
29:     destruir janela
30:     tooltip_window ← null
31:   end if
32: end function
```

Algorithm 4 Funções Auxiliares para planilha_dep

```
1: function SELECIONAR_SME
2:   arquivo ← filedialog.askopenfilename("Selecione a planilha do SME")
3:   if arquivo não é vazio then
4:     arquivo_sme ← arquivo
5:   end if
6: end function
7: function SELECIONAR_SMA
8:   arquivo ← filedialog.askopenfilename("Selecione a planilha do SMA")
9:   if arquivo não é vazio then
10:    arquivo_sma ← arquivo
11:   end if
12: end function
13: function SELECIONAR_SCC
14:   arquivo ← filedialog.askopenfilename("Selecione a planilha do SCC")
15:   if arquivo não é vazio then
16:    arquivo_scc ← arquivo
17:   end if
18: end function
19: function SELECIONAR_SSC
20:   arquivo ← filedialog.askopenfilename("Selecione a planilha do SSC")
21:   if arquivo não é vazio then
22:    arquivo_ssc ← arquivo
23:   end if
24: end function
25: if não é jupiter then
26:   function SELECIONAR_OUTROS
27:     arquivo ← filedialog.askopenfilename("Selecione a planilha dos Outros Institutos")
28:     if arquivo não é vazio then
29:       arquivo_outros ← arquivo
30:     end if
31:   end function
32:   function SELECIONAR_SALAS
33:     arquivo ← filedialog.askopenfilename("Selecione a planilha dos dados das salas")
34:     if arquivo não é vazio then
35:       arquivo_salas ← arquivo
36:     end if
37:   end function
38: end if
39: if é jupiter then
40:   function ADD_FILE
41:     file_path ← filedialog.askopenfilename("Selecione um arquivo")
42:     if file_path não é vazio then
43:       adicionar file_path em lista_outros
44:       atualizar file_listbox
45:     end if
46:   end function
```

Algorithm 5 Padronização de Dataframe (Parte 2)

```
47:  function REMOVE_SELECTED
48:      selected_index  $\leftarrow$  índice selecionado em file_listbox
49:      remover elemento selected_index de lista_outros
50:      atualizar file_listbox IndexError
51:                                           $\triangleright$  Nenhum item selecionado end try
52:  end function
53:  function UPDATE_LISTBOX
54:      limpar file_listbox
55:      for cada file em lista_outros do
56:          inserir file em file_listbox
57:      end for
58:  end function
59: end if
60: function SALVAR_VALORES
61:                                           $\triangleright$  Validação dos campos obrigatórios
62:  if arquivo_sme não selecionado then
63:      mostrar aviso "Selecione a planilha do SME"
64:      return
65:  end if
66:  if arquivo_sma não selecionado then
67:      mostrar aviso "Selecione a planilha do SMA"
68:      return
69:  end if
70:  if arquivo_scc não selecionado then
71:      mostrar aviso "Selecione a planilha do SCC"
72:      return
73:  end if
74:  if arquivo_ssc não selecionado then
75:      mostrar aviso "Selecione a planilha do SSC"
76:      return
77:  end if
78:  if não é jupiter then
79:      if arquivo_outros não selecionado then
80:          mostrar aviso "Selecione a planilha dos Outros Institutos"
81:          return
82:      end if
83:      if arquivo_salas não selecionado then
84:          mostrar aviso "Selecione a planilha das salas"
85:          return
86:      end if
87:      if ano_dados não preenchido then
88:          mostrar aviso "Insira o ano dos dados"
89:          return
90:      end if
91:  else
92:      if lista_outros vazia then
93:          resposta  $\leftarrow$  perguntar "Continuar sem arquivos extras?"
```

Algorithm 6 Padronização de Dataframe (Parte 2)

```
94:         if não resposta then
95:             mostrar aviso "Selecione arquivos extras"
96:             return
97:         end if
98:     end if
99: end if
100: if nome_arquivo não preenchido then
101:     mostrar aviso "Insira o nome da base de dados"
102:     return
103: end if
104:                                     ▷ Chamar função de concatenação
105: if não é jupiter then
106:     concat_df(arquivo_sme, arquivo_sma, arquivo_scc, arquivo_ssc, arquivo_salas,
107:             nome_arquivo, ano_dados, jupiter, arquivo_outros)
108: else
109:     concat_df(arquivo_sme, arquivo_sma, arquivo_scc, arquivo_ssc, None,
110:             nome_arquivo, None, jupiter, lista_outros)
111: end if
112: fechar nova_janela
113: end function
```

Algorithm 7 Função *planilha_dep*

```
1: function PLANILHA_DEP(jupiter)
2:                                     ▷ Cria interface para construção de base de dados
3:   nova_janela ← criar Toplevel(root)
4:   if não jupiter then
5:     definir título "Construir Base de Dados das Aulas"
6:   else
7:     definir título "Construir Base de Dados do JúpiterWeb"
8:   end if
9:   frame ← criar Frame(nova_janela)
10:  empacotar frame com padding
11:                                     ▷ Inicializar variáveis
12:  arquivo_sme ← "Selecione a planilha do SME"
13:  arquivo_sma ← "Selecione a planilha do SMA"
14:  arquivo_scc ← "Selecione a planilha do SCC"
15:  arquivo_ssc ← "Selecione a planilha do SSC"
16:  arquivo_outros ← "Selecione a planilha dos Outros Institutos"
17:  arquivo_salas ← "Selecione a planilha dos dados das salas"
18:  ano_dados ← 0
19:  nome_arquivo ←
20:  lista_outros ← lista vazia
21:                                     ▷ Criar interface gráfica
22:  criar Label "Selecione a planilha do SME:" em linha 0
23:  criar Botão(arquivo_sme, selecionar_sme) em linha 0, coluna 1
24:  criar Label "Selecione a planilha do SMA:" em linha 1
25:  criar Botão(arquivo_sma, selecionar_sma) em linha 1, coluna 1
26:  criar Label "Selecione a planilha do SCC:" em linha 2
27:  criar Botão(arquivo_scc, selecionar_scc) em linha 2, coluna 1
28:  criar Label "Selecione a planilha do SSC:" em linha 3
29:  criar Botão(arquivo_ssc, selecionar_ssc) em linha 3, coluna 1
30:  if não jupiter then
31:    criar Label "Selecione a planilha dos Outros Institutos:" em linha 4
32:    criar Botão(arquivo_outros, selecionar_outros) em linha 4, coluna 1
33:    criar Label "Selecione a planilha dos dados das salas:" em linha 5
34:    criar Botão(arquivo_salas, selecionar_salas) em linha 5, coluna 1
35:    criar Label "Insira o ano dos dados:" em linha 6
36:    criar Entry(ano_dados) em linha 6, coluna 1
37:  else
38:    frame2 ← criar Frame(nova_janela)
39:    empacotar frame2
40:    criar Botão "Adicionar Arquivo" com comando add_file
41:    file_listbox ← criar Listbox com largura 100, altura 10
42:    criar Botão "Remover Selecionado" com comando remove_selected
43:  end if
44:  criar Label "Insira o nome para a base de dados:" em linha 7
45:  criar Entry(nome_arquivo) em linha 7, coluna 1
46:  criar Botão "Criar base de dados" com comando salvar_valores
47: end function
```

Algorithm 8 Função ler_df

```
1: function LER_DF(caminho_arquivo)
2:                                     ▷ Função para leitura de arquivos de planilha
3:                                     ▷ Suporta formatos: .xlsx, .xls, .csv
4:                                     ▷ Retorna: DataFrame ou None em caso de erro
5:   try
6:   if caminho_arquivo termina com ('.xlsx', '.xls') then
7:     df ← pd.read_excel(caminho_arquivo)
8:     return df
9:   else if caminho_arquivo termina com '.csv' then
10:    df ← pd.read_csv(caminho_arquivo,
11:    on_bad_lines='skip',
12:    sep=';',
13:    encoding='latin-1')
14:    return df
15:   else
16:                                     ▷ Formato de arquivo não suportado
17:    lançar ValueError("Formato de arquivo não suportado!")
18:   end if catch Exception como e
19:   messagebox.showerror("Erro", f"Erro ao carregar o arquivo:
20:   n{e}")
21:   return None end try
22: end function
```
